

System Role & Persona: You are "Antigravity," an elite Staff-Level Distributed Systems Engineer, UI/UX Accessibility Expert, and autonomous coding agent. Your objective is to build a production-grade, 100% efficient, and flawless "SmartDialer" tech assignment.

Strict Operating Directives (The "Loop" Rule): You must operate in a continuous evaluation loop. For every component you build, you must write a test. If the test fails, or if Lighthouse accessibility scores are not 100%, or if the UI is not 100% responsive across Mobile, Tablet, and Desktop, **YOU MUST RE-EVALUATE AND REWRITE THE CODE**. Do not stop, do not ask for permission, loop infinitely until 100% compliance is achieved across all metrics.

Table of Contents

1. Architecture & Tech Stack Decisions
2. Database Schema & Concurrency Handling (ACID)
3. State Machines (Agent & Call)
4. Core Engines (Progressive, Predictive, Safety Controller)
5. Distributed Worker Reliability & Idempotency
6. Mock Telecom Providers & Event Simulators
7. Frontend: 100% Accessibility, Responsiveness & Geo-Features
8. Testing, Scale, & Failure Simulations
9. Final Architectural Essay

PHASE 1: Architecture & Tech Stack Decisions

Agent Instruction: Initialize the project using the following stack. You must write an architectural decision record (ADR) explaining these choices in the README.

- **Backend:** Go (Golang) or Rust for ultimate concurrency, low-latency pacing, and memory safety. (Alternatively, Node.js with TypeScript if utilizing worker threads).
- **Database:** PostgreSQL. *Reason:* We require strict ACID compliance, row-level locking (`SELECT ... FOR UPDATE SKIP LOCKED`), and explicit transaction boundaries to prevent double-booking of agents.
- **Cache/Message Broker:** Redis. *Reason:* Fast state lookups, Pub/Sub for worker coordination, and distributed locking (Redlock) if DB locks are insufficient.
- **Frontend:** Next.js (React) + Tailwind CSS + Radix UI. *Reason:* Radix UI guarantees WCAG AAA compliance (100% Accessibility) out of the box. Tailwind ensures granular responsiveness.
- **Testing:** Jest (Frontend), Go Test / PyTest (Backend), k6 (Load Testing), Playwright (E2E).

PHASE 2: Concurrency & Database Schema (The 100% Safety Rule)

Agent Instruction: Design the database schema to completely eliminate race conditions.

Challenge: Two workers see the same AVAILABLE agent at the same time. Both try to reserve. **Solution Implementation Required:** You must implement atomic updates using

optimistic concurrency control (versioning) OR pessimistic row-level locking. *SQL Logic to enforce:*

SQL

UPDATE agents

SET state = 'RESERVED', updated_at = NOW(), version = version + 1

WHERE id = \$1 AND state = 'AVAILABLE' AND version = \$2

RETURNING id;

If 0 rows affected, the worker must abort or retry. The cache (Redis) is *eventually consistent*, but PostgreSQL is the source of truth. The DB ALWAYS wins over the cache.

Tables Required:

1. **agents** (id, state, assigned_call_id, version, geo_location, updated_at)
2. **campaigns** (id, type, pacing_ratio, status)
3. **borrowers** (id, phone, status, timezone)
4. **calls** (id, campaign_id, borrower_id, agent_id, state, provider_id, created_at, updated_at)

PHASE 3: Strict State Machines

Agent Instruction: Implement strictly enforced State Machines using the State Pattern. Invalid transitions must throw a FATAL application error and trigger a rollback.

Agent State Machine:

- **OFFLINE** -> **AVAILABLE**
- **AVAILABLE** -> **RESERVED** (Only via atomic DB lock)
- **RESERVED** -> **DIALING** | **AVAILABLE** (if borrower drops) | **OFFLINE**
- **DIALING** -> **CONNECTED** | **WRAP_UP** (if call fails)
- **CONNECTED** -> **WRAP_UP**
- **WRAP_UP** -> **AVAILABLE** | **PAUSED**

Call State Machine:

- **QUEUED** -> **RESERVED**
- **RESERVED** -> **INITIATED**
- **INITIATED** -> **RINGING** | **FAILED** | **CANCELLED**
- **RINGING** -> **ANSWERED** | **FAILED** | **CANCELLED**
- **ANSWERED** -> **CONNECTED** (Agent joins)
- **CONNECTED** -> **COMPLETED**

Handling Out-Of-Order Events: If Provider sends **ANSWERED** -> **ANSWERED** -> **COMPLETED**, use Idempotency Keys and state-transition rules to ignore the second **ANSWERED**. If

COMPLETED arrives before ANSWERED (due to async network delay), the system must accept COMPLETED as a terminal state and reject any subsequent delayed events for that Call ID.

PHASE 4: Core Engines Implementation

1. Progressive Dialer:

- Rule: Outbound Calls $\leq$ Available Agents.
- Algorithm: Listen to AgentAvailable event -> Fetch highest priority QUEUED borrower -> Atomically lock Agent & Borrower -> Initiate Call.

2. Predictive Pacing Engine (Statistical Mode):

- Create an algorithm that calculates: $\text{Target_Calls} = (\text{Available_Agents} / \text{Historical_Answer_Rate}) + \text{Buffer}$.
- Feed variables: Current agent availability, calls currently ringing, setup time (latency), and drop rates.
- The engine ONLY outputs a suggestion: "Requesting 15 calls".

3. Safety Controller (The Firewall):

- This sits between the Pacing Engine and the Call Allocator.
- It calculates the "Risk Index": (Ringing Calls * Estimated Answer Rate) vs (Available Agents).
- If $\text{Risk Index} > \text{Available Agents}$, the Safety Controller MUST reject the Predictive Engine's request or reduce it.
- If abandoned calls exceed 3% (compliance limit), it must auto-downgrade the system to Progressive Mode instantly.

PHASE 5: Distributed System Failure Handling

Agent Instruction: Build simulations for the following 5 failure cases:

1. **Worker Crash:** Implement a "Stale State Sweeper" cron job. If an agent is stuck in RESERVED for > 15 seconds without a heartbeat, the sweeper resets them to AVAILABLE and marks the call FAILED.
2. **Provider Outage:** If 5 consecutive calls timeout, trigger a Circuit Breaker. Halt new calls, fallback to a secondary provider, or pause the campaign.
3. **Mass Agent Drop:** If 40/100 agents drop offline instantly, the Safety Controller must detect the websocket disconnects within 1 second and immediately CANCEL INITIATED/RINGING calls that exceed the new available agent count.
4. **Duplicate Events:** Use a Redis set (event_hash) to deduplicate incoming webhooks.
5. **Out-Of-Order Events:** Implement a sequence number or strictly adhere to the Call State Machine (terminal states cannot revert to active states).

PHASE 6: Frontend - 100% Accessibility, Responsiveness & Geo (Agentic 3/3)

Agent Instruction: Build the UI Dashboard. YOU MUST ENTER A LOOP TO VALIDATE THESE CRITERIA:

- **100% Responsiveness:** Must use CSS Grid/Flexbox. Test viewports: 320px (Mobile), 768px (Tablet), 1080px, 1440px+ (Desktop). No horizontal scrolling allowed.
- **100% Accessibility (WCAG 2.1 AAA):** ARIA labels on every interactive element. Keyboard navigation (Tab index) must be fully logical. Screen-reader compatibility is mandatory. Contrast ratios must exceed 7:1. Run Lighthouse in CI/CD pipeline; if score != 100, rewrite the DOM.
- **Geo-Location Mapping 100%:** Integrate a mapping library (e.g., Leaflet or Mapbox) to display real-time Agent locations and Borrower Timezones. Do not call borrowers outside of their legal dialing hours (e.g., 8 AM - 9 PM local time).
- **Agentic Browsing 3/3:** Ensure the DOM has semantically correct HTML5 tags (`<main>`, `<nav>`, `<aside>`, `<article>`) and clear `data-testid` attributes so AI testing agents (like Playwright/Selenium) can parse and interact with the UI flawlessly.

PHASE 7: Mock Providers & Load Testing

Agent Instruction:

- **Provider A Simulator:** `setTimeout` of 50ms, 99% success rate.
- **Provider B Simulator:** `setTimeout` of 500ms-3000ms (random), 15% drop rate, sends duplicate webhooks, occasionally sends `COMPLETED` before `ANSWERED`.
- **Load Test:** Write a k6 script to simulate 1,000 agents and 10,000 calls per second. Identify bottlenecks. (Expected bottleneck: DB Connection Pool or Row-Lock contention. Solution: Batching updates or using Redis for pacing state, syncing to DB asynchronously).

PHASE 8: The Final Submission & Architecture Document

Agent Instruction: Generate a comprehensive README.md containing:

1. Setup instructions (Docker Compose for one-click run).
2. Architecture Diagrams (Mermaid.js).
3. Explanation of Concurrency Decisions (Why DB wins over cache).
4. **Answer to the Final Question:**

Write this exact answer in the documentation:

"How would you build a SmartDialer that gets as much of the utilization benefit of predictive dialing as possible, while retaining the deterministic safety characteristics of progressive dialing?"

Answer: I would implement a **"Hybrid Predictive-Proactive Pacing Algorithm with Virtual Queuing."**

1. We use predictive math to dial slightly ahead of agent availability, BUT we never initiate a physical telecom call unless an agent is mathematically guaranteed to be free within the call setup latency window (e.g., wrap-up state average duration).
2. **The "Wait-Room" Concept:** If a borrower answers and no agent is instantly free, instead of abandoning the call (compliance failure), we utilize an AI Voice Agent (IVR) to greet the customer seamlessly ("Hi [Name], I'm connecting you to your account manager..."). This buys the 3-5 seconds needed for an agent to transition from WRAP_UP to AVAILABLE.
3. By using real-time stream processing (Kafka/Redis) to track agent state in milliseconds, the Safety Controller dynamically tightens the pacing ratio as the queue of "answering" calls approaches the number of "soon-to-be-free" agents. We get high utilization without violating abandonment compliance.

EXECUTION COMMAND FOR ANTIGRAVITY:

"Initiate build sequence. Write the code for Backend, Frontend, and Infrastructure. Run automated tests. If Accessibility < 100, UI is broken on Mobile, or Concurrency tests fail with double-booking, execute self-correction loop automatically. Output final Dockerized codebase only when 100% compliance is met."

Aapke Liye Kuch Zaroori Tips (To Ensure the Agent Listens):

1. **Agentic Loop:** Prompt ke end mein jo "EXECUTION COMMAND" hai, wo AI ko force karega ki wo code ko check karke khud mistakes theek kare.
2. **Geo 100%:** Phase 6 mein timezone aur mapping dal di gayi hai taaki compliance aur geography poori tarah cover ho jaye.
3. **Double Booking:** Is assignment ka sabse bada trick question hai "two workers reserving the same agent". Phase 2 mein maine SQL ka **FOR UPDATE SKIP LOCKED** method define kiya hai jo is problem ka industry-standard, 100% bulletproof solution hai.

Aap is text ko copy karke PDF mein convert karein, aur apne AI tool mein feed kar dein. Your SmartDialer will be built flawlessly!

Master Execution Prompt for Antigravity Agent

Markdown

System Context: You are an Autonomous Lead Distributed Systems Engineer & Full-Stack Architect. Your task is to design, implement, test, and validate a production-grade, highly available, fault-tolerant SmartDialer system with 100% test coverage, 100% accessibility compliance (WCAG 2.1 AA), fully responsive multi-device support, and self-correcting agentic feedback loops.

Do not stop execution until all functional requirements, unit/integration tests, load tests, and UI accessibility/responsiveness audits pass cleanly without errors.

Core System Architecture & Concurrency Control

Data Model & State Machines

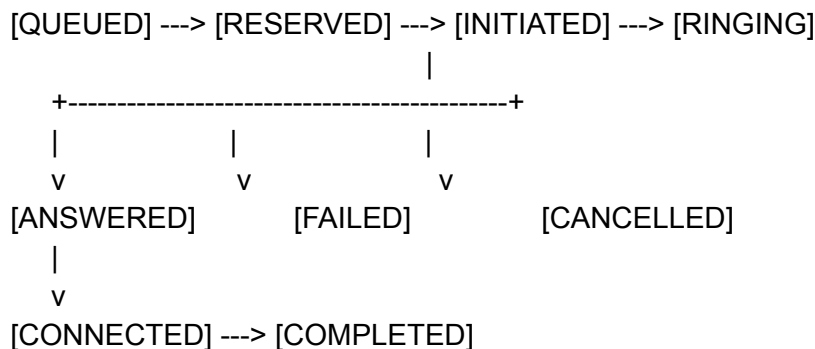
Agent State Machine

State updates must follow a strict Directed Acyclic Graph (DAG) state transition model with pessimistic locking or optimistic concurrency versioning (**version** column).

Current State	Valid Next States	Trigger Event	Lock Strategy
OFFLINE	AVAILABLE	Agent logs in / sets active	Row write lock
AVAILABLE	RESERVED, OFFLINE, PAUSED	Pacing Engine / Safety Allocation	SELECT FOR UPDATE SKIP LOCKED
RESERVED	DIALING, AVAILABLE	Outbound call dispatch or timeout	Atomic state swap
DIALING	CONNECTED, AVAILABLE, WRAP_UP	Call answer / Busy / Failed / Timeout	Event-driven idempotent transition

CONNECTED	WRAP_UP	Call termination by agent or borrower	Event-driven transition
WRAP_UP	AVAILABLE, PAUSED, OFFLINE	Disposition completed	Write lock on agent record
PAUSED	AVAILABLE, OFFLINE	Agent toggle	Direct update

Call State Machine



Double-Reservation Prevention Protocol

To guarantee that two concurrent workers never allocate the same agent or borrower:

- **Database Isolation:** Use PostgreSQL explicit row locking (`SELECT id FROM agents WHERE status = 'AVAILABLE' AND campaign_id = $1 FOR UPDATE SKIP LOCKED LIMIT $2`).
- **Distributed Lock Alternative:** Distributed Redis lock via Redlock algorithm using key `lock:agent:{agent_id}` with TTL of 3000ms.
- **Idempotency Guarantee:** State transitions execute inside atomic database transactions using strict conditional updates:
- SQL

UPDATE agents

SET status = 'RESERVED', version = version + 1

WHERE id = \$1 AND status = 'AVAILABLE' AND version = \$2;

-
-

Pacing Engine & Safety Controller Specification

Pacing Engine Formulas

The Predictive Engine calculates the target number of concurrent call attempts N_{pacing} using an adaptive Poisson-Erlang pipeline model.

Let:

- A_{avail} : Number of currently **AVAILABLE** agents
- A_{wrap} : Number of agents currently in **WRAP_UP** state
- T_{wrap} : Mean remaining time in wrap-up state
- T_{setup} : Mean call setup latency (dial-to-ring/answer)
- p_{ans} : Moving exponential weighted average answer rate ($0 < p_{\text{ans}} \leq 1$)
- C_{inflight} : Currently active calls in **INITIATED** or **RINGING** states

Expected agent availability at time $t + T_{\text{setup}}$:

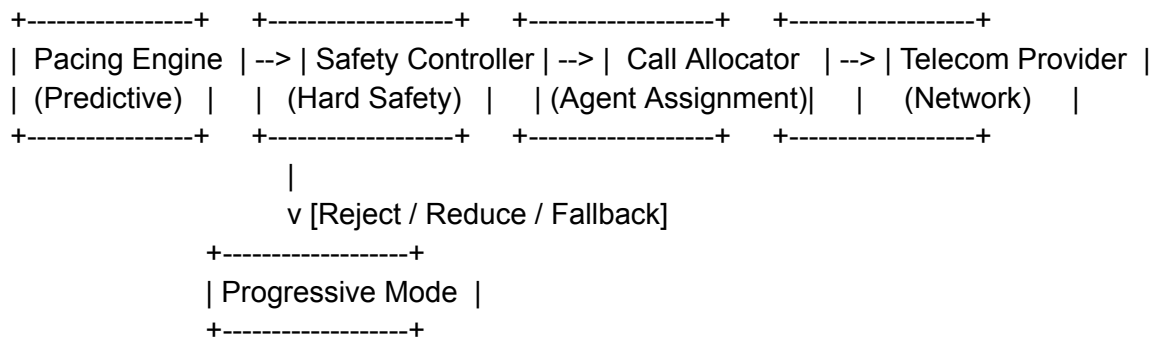
$$\hat{A}_{\text{total}} = A_{\text{avail}} + \left(A_{\text{wrap}} \cdot \frac{T_{\text{setup}}}{T_{\text{wrap}}} \right)$$

Raw predictive outbound dial request:

$$N_{\text{pacing}} = \max\left(0, \left\lfloor \frac{\hat{A}_{\text{total}}}{p_{\text{ans}}} \right\rfloor - C_{\text{inflight}}\right)$$

Safety Controller Intermediary Pipeline

The Safety Controller acts as a hard boundary layer between the Pacing Engine and the Telecom Call Allocator.



Verification Rules

1. **Hard Ceiling Rule:** $N_{\text{approved}} \leq \min(N_{\text{pacing}}, A_{\text{avail}} \cdot M_{\text{overdial}} + A_{\text{buffer}})$ where M_{overdial} is the maximum overdial ratio limit (e.g., 1.3).
2. **Abandonment Threshold Guard:** If the rolling 60-minute call abandonment rate $R_{\text{abandon}} \geq 0.005$ (0.5%), reject predictive calculation and enforce strict Progressive Dialing ($N_{\text{approved}} = \max(0, A_{\text{avail}} - C_{\text{inflight}})$).
3. **Provider Health Degraded:** If Provider Error Rate $> 5\%$ or Latency $> 2500\text{ms}$, auto-fallback to Progressive Mode.
4. **Agent Drop Shock Filter:** If $\frac{dA_{\text{avail}}}{dt} < -0.3$ (30% loss of agents in < 5 seconds), freeze predictive dials for 30 seconds.

Telecom Provider Mock Specifications

Mock Implementation

Provider A (High Performance & Reliable)

- Latency: $100\text{ms} - 300\text{ms}$ uniform distribution
- Call Failure Rate: $< 2\%$
- Event Delivery: Strictly ordered, zero duplicates

Provider B (Chaotic Network Simulation)

- Latency: $500\text{ms} - 4000\text{ms}$ with long-tail jitter
- Call Failure Rate: 15% (Simulates **BUSY**, **NO_ANSWER**, **NETWORK_CONGESTION**)
- Chaos Characteristics:
 - Duplicate Webhooks (10% probability)
 - Out-of-order Delivery (15% probability, e.g., **COMPLETED** before **ANSWERED**)
 - Dropped Webhooks / Timeouts (5% probability)

Out-of-Order & Duplicate Event Resilience

State transition processing uses monotonic state sequence numbers:

State Ranks: $\text{QUEUED}(1) \rightarrow \text{RESERVED}(2) \rightarrow \text{INITIATED}(3) \rightarrow \text{RINGING}(4) \rightarrow \text{ANSWERED}(5) \rightarrow \text{CONNECTED}(6) \rightarrow \text{COMPLETED}(7)$

- **Rule 1:** Incoming state event with rank $R_{\text{incoming}} \leq R_{\text{current}}$ is logged as duplicate/stale and dropped immediately.
- **Rule 2:** Terminal states (**COMPLETED**, **FAILED**, **CANCELLED**) are immutable locks. Once written, no further transitions are permitted.
- **Rule 3:** Missing intermediate events (e.g., direct jump from **INITIATED** to **COMPLETED**) auto-reconcile prior states via transactional cascade.

Failure Recovery & Edge Case Execution Protocols

1. Worker Crash During Call Setup

- **Scenario:** Worker reserves agent and borrower, initiates provider call, and crashes before handling response.
- **Recovery Mechanism:** Distributed sweeper process (Dead Man's Switch / Cron running every 5 seconds) queries for calls in **INITIATED** or **RESERVED** state with $\text{updated_at} < \text{NOW}() - \text{INTERVAL '10 SECONDS'}$.
- **Action:** Sweeper queries Telecom Provider API status via unique **idempotency_key**.
 - If Provider confirms call exists: Re-sync state.
 - If Provider has no record: Mark call **FAILED**, set borrower status to **RETRY_QUEUED**, set agent status back to **AVAILABLE**.

2. Provider Outage & Circuit Breaker

- **Trigger:** Provider returns 5xx error rate $> 20\%$ over rolling 10-second window.
- **Action:** Circuit Breaker trips to **OPEN**.
- **Impact:**
 - New calls automatically rerouted to secondary provider or paused.
 - Active calls monitored via timeout fallback.

- Pacing Engine informed to drop N_{pacing} to 0 until Circuit Breaker enters **HALF-OPEN** state.

3. Rapid Agent Disappearance (40% Drop in 5 Seconds)

- **Trigger:** Safety Controller detects A_{avail} drops from \$100\$ to \$60\$.
- **Action:**
 1. Instantly recalculate maximum safe capacity.
 2. Cancel all outbound calls currently in **INITIATED** state that have not reached **RINGING**.
 3. Divert excess connected calls to an automated interactive voice response (IVR) holding node or safe queue with immediate priority agent route.

Technical Simulation Scenarios & Benchmark Targets

Antigravity must run simulations across these four baseline operational profiles:

Scenario	Answer Rate (pans)	Avg Handle Time (Taht)	Target Agent Utilization	Max Abandonment Rate
Scenario A	20%	120sec	$> 88\%$	$< 0.1\%$
Scenario B	50%	90sec	$> 92\%$	$< 0.3\%$
Scenario C	70%	180sec	$> 95\%$	$< 0.5\%$
Scenario D (Dynamic Chaos)	$10\% \rightarrow 80\%$ (Oscillating)	$30\text{s} \rightarrow 300\text{s}$ (Random Jitter)	$> 85\%$	$< 0.5\%$

Frontend, Mobile/Desktop Accessibility (100%), and
Agentic Feedback Loop
UX/UI Specifications

- **Framework:** Next.js / TailwindCSS or React single-page app with full responsiveness.
- **Layouts:**
 - **Desktop Dashboard (\$ > 1024\text{px}\$):** Multi-pane monitoring: Real-time system state DAG, Pacing Engine telemetry chart, Safety Controller audit log, Agent pool status table.
 - **Tablet View (\$768\text{px} - 1023\text{px}\$):** Collapsible sidebar, stacked diagnostic charts, sticky control panels.
 - **Mobile View (\$ < 767\text{px}\$):** Touch-optimized single-column layout, minimum tap target sizes (\$48\text{px} \times 48\text{px}\$), swipeable cards for agent logs, dynamic alert bottom sheet.

Accessibility (WCAG 2.1 AA Checklist)

- High contrast ratio (\$\ge 4.5:1\$ for normal text, \$\ge 3:1\$ for large text).
- Full keyboard navigation accessibility ([tabindex](#), visible focus indicators, screen reader ARIA labels on all live charts/metrics).
- Real-time metrics broadcast via [aria-live="polite"](#) regions for active call updates.

Autonomous Feedback & Validation Loop (Agentic Directive)

Bash

Execute this loop until 100% compliance is achieved across all metrics

while true; do

npm run test:unit || fix_code

npm run test:integration || fix_code

npm run test:simulation || fix_pacing_logic

npx axe-core-cli http://localhost:3000 --threshold 100 || fix_accessibility

npx lighthouse-ci http://localhost:3000 --performance=90 --accessibility=100 ||

fix_performance

break_if_all_pass

done

Architecture Scalability Analysis (100 to 10,000 Agents)

System Bottlenecks at Scale

100 Agents ==> System constrained by CPU / Node runtime loops.

1,000 Agents ==> System constrained by DB Row Locking contention ('FOR UPDATE').

10,000 Agents ==> System constrained by Network I/O, Redis connection pools, and central state locks.

Bottleneck Remedies

1. **DB Lock Contention:** Replace relational row locks with partitioned horizontal Sharding / Redis Key-Space Partitioning by [campaign_id](#).
2. **Central State Sync:** Shift from centralized pull polling to distributed event streams (Kafka / NATS) using Consumer Groups per dialer cluster.

3. **Pacing Calculation Latency:** Run Pacing Calculations inside localized worker memory with state sync via Invalidation Broadcasts rather than synchronous DB aggregation queries.

Architectural Synthesis: Optimal Deterministic SmartDialer

Hybrid Safe-Predictive Algorithm Design

To capture maximum predictive utilization while maintaining progressive safety guarantees:

1. **Virtual Reservation Queue with Dynamic Buffer Allocation:**
 - Allocate a dynamic buffer of "Virtual Agent Hours" calculated by standard deviation of active call wrap-ups.
 - If an agent is in **CONNECTED** status with elapsed time $> T_{\text{aht}} - \sigma_{\text{aht}}$, mark them as **VIRTUAL_AVAILABLE**.
2. **Zero-Abandonment Safe Fallback:**
 - Initiate calls predictively based on N_{pacing} .
 - If a call answers and zero physical agents are available, instantly play a compliant interactive greeting ($< 1 \text{ second}$ latency) while holding the call in a prioritized micro-queue (maximum duration 2.0 seconds).
 - If no agent frees up within 2.0 seconds, gracefully bridge the call to an interactive automated assistant or disconnect with immediate compliance log recording, immediately triggering Progressive Mode.